

Centralised logging and incident reconstruction

Follow-along: collect what four machines record, then answer for one of them after its own copy is gone

Overview

Definition: Log message

A line a program hands to the operating system's logging service, describing something the program just did. It carries the text of the event, the name of the program, the time, and two numbers that classify it: the *facility*, which says what kind of subsystem produced it, and the *severity*, which says how urgent it is. RFC 5424 section 6.2.1 carries both tables.

A machine's record of what happened to it is a file on that machine. That is enough until the thing that happened is somebody taking control of the machine, because then the record and the person who would prefer it did not exist are on the same disk, and `rm` is one command. Centralised logging is the arrangement built for this: each machine sends a copy of every **log message** it produces to a second machine over the network, as it produces it, so the copy is already somewhere else before anybody thinks to delete the original.

This lab builds that arrangement across four machines and then tests it the only way that means anything. You configure the sending, the receiving, one destination file per source machine and a rotation policy; then an intrusion runs across three of the machines and the local log on the one it happened to is deleted, and every question you are asked afterwards has to be answered from the collector alone. What the lab does not cover: it sends messages in the clear, so it does nothing to stop somebody on the segment reading or forging them, and it does nothing to prove that the collector's own files have not been edited since they were written. Transport encryption (RFC 5425) and signing are separate mechanisms and neither is configured here.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Host and service management**, then **Centralised logging and incident reconstruction**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image; later spawns take seconds.
- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It sends a message from each of the three machines and reports whether it

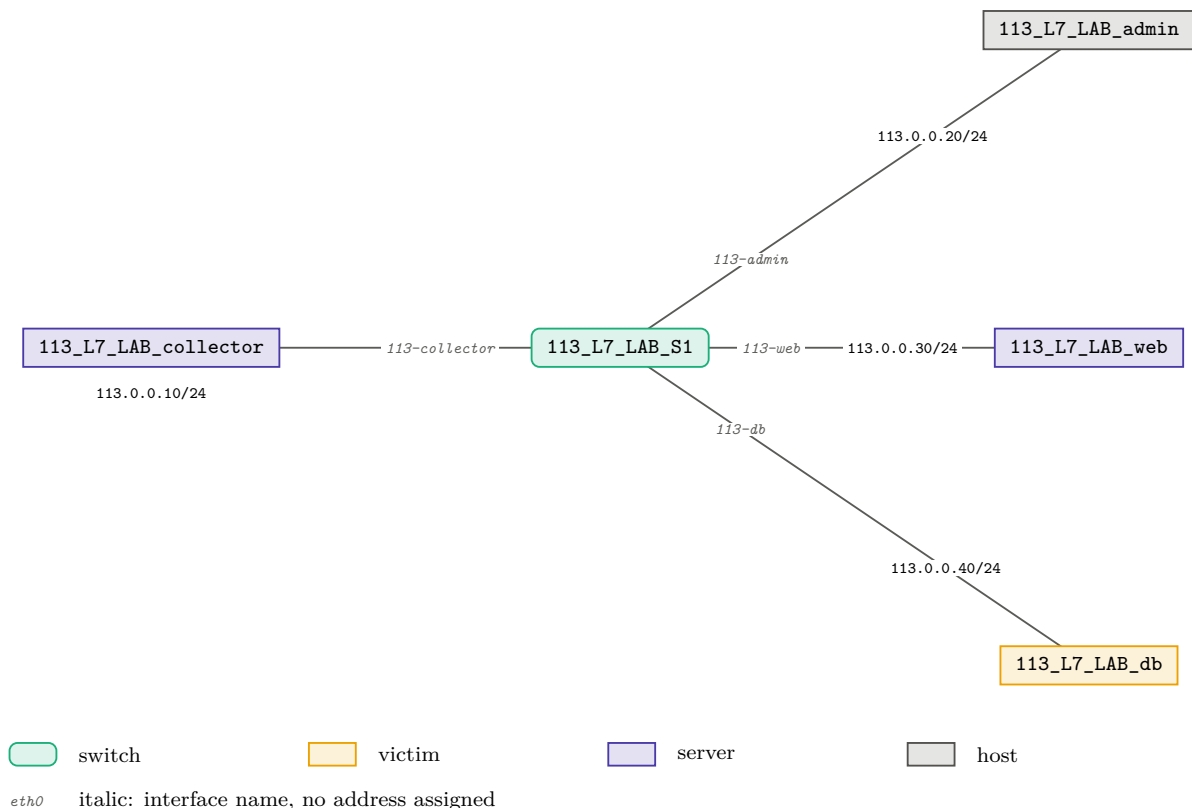
reached the collector and what the collector recorded beside it. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** lists every node (collector, admin, web, db) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Run the incident** runs an intrusion across the three machines and then deletes one machine's own log file. Do not choose it until Part 4 tells you to. It checks first that the collector is recording, and if it is not it stops and deletes nothing, because otherwise there would be no copy of what happened left anywhere.
- **Reset to baseline** deletes every rule you wrote, empties the collector, and puts the deleted log file back, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

Four machines share one switched segment. There is no router and nothing to route to: every machine is one hop from every other, so a message that does not arrive at the collector failed for a reason in the logging configuration and never for a reason in the network.



Machine	Address	Runs	Its part in the lab
collector	113.0.0.10	rsyslog	the log server; at spawn it listens on no port
admin	113.0.0.20	rsyslog, sshd	the administrator's workstation
web	113.0.0.30	rsyslog, lighttpd	the public web service
db	113.0.0.40	rsyslog, sshd	the sensitive host

Every machine already runs `rsyslog` 8.2310 and writes what it produces to `/var/log/messages` on its own disk. Nothing is sent anywhere. `db` and `admin` accept SSH logins with a password; `web` serves one page over HTTP and sends both of `lighttpd`'s logs through `syslog` rather than to files of its own. The tools you need are already on every machine: `logger` to produce a message by hand, `tcpdump` to read what crosses the wire, and `logrotate` on the collector.

Assessment

There are nineteen questions, placed at the step that produces the evidence for each. Answer all nineteen in a separate document. Several of them ask for a value you can only read off your own screen; record it when the step tells you to, because Parts 3 and 4 change what is on the disk.

Part 1: what a machine records about itself

Definition: Facility

A number from 0 to 23 saying which kind of subsystem produced a message, with a conventional name for each. `auth` is 4 and covers authentication; `daemon` is 3 and covers system services; `local0` through `local7` are 16 to 23 and are reserved for local use, which is what makes them safe to use for a message you produce by hand. RFC 5424 requires the value to be in the range 0 to 23 and presents the names in Table 1 as informational rather than normative, so a program chooses its own facility and nothing enforces the choice.

Definition: Severity

A number from 0 to 7 saying how urgent a message is, where *smaller means more urgent*: `emerg` 0, `alert` 1, `crit` 2, `err` 3, `warning` 4, `notice` 5, `info` 6, `debug` 7. So `err` is more severe than `warning`, and `debug` is the least severe of the eight.

Open a shell on the sensitive host and look at what it has already written about itself. The **facility** and **severity** of each line were chosen by whichever program produced it.

```
# 113_L7_LAB_db: the sensitive host
docker exec -it 113_L7_LAB_db bash
```

```
# 113_L7_LAB_db: read the machine's own record of itself
tail -20 /var/log/messages
```

You will see lines from `sshd` and from `rsyslogd` itself. Now add one of your own. `logger` hands a message to the same service every other program uses, and `-p` sets the facility and the severity:

```
# 113_L7_LAB_db
logger -p local5.notice "first message from me"
logger -p auth.warning "second message from me"
tail -4 /var/log/messages
```

Question 1. Both of your messages are in the file. State, for each of the two, whether you can read its facility and its severity back out of the line as it was written, and name the four things the line does record.

Definition: Selector

The `facility.severity` pair at the start of a classic syslog rule, which decides whether that rule's destination gets a copy of a message. `*` matches any facility or any severity, so `*.*` is every message, and this is the rule every machine in the lab arrives with. In `rsyslog` and in the `syslogd` syntax it inherits, a severity in a selector means *that severity and every more severe one*, so `auth.warning` selects `auth.warning`, `auth.err`, `auth.crit` and up, and does not select `auth.info`.

Definition: rsyslog configuration objects

`rsyslog` is configured out of five kinds of statement. A `module()` loads code: `imuxsock` reads `/dev/log`, `imtcp` listens on a TCP port, and the `om` modules write messages out. An `input()` turns a loaded input module into something actually bound and listening. An `action()` is one destination a message is written to. A `template()` defines the text of a line or the name of a file, out of the message's own fields. A `ruleset()` is a named group of rules that only the inputs bound to it feed.

Each machine's base configuration is in `/etc/rsyslog.conf` and you do not edit it. It loads one **module**, `imuxsock`, and writes everything to one file. Everything you add in this lab goes in a file of its own under `/etc/rsyslog.d/`, which the base configuration includes. Look at the one rule that is there:

```
# 113_L7_LAB_db
cat /etc/rsyslog.conf
```

Write a second destination that takes only the more severe authentication messages, using a **selector** rather than `*.*`:

```
# 113_L7_LAB_db: a rule of your own, in a drop-in file
cat > /etc/rsyslog.d/30-authwarn.conf <<'EOF'
auth.warning    /var/log/auth-warn.log
EOF
chmod 644 /etc/rsyslog.d/30-authwarn.conf
```

A configuration change needs the daemon replaced, not signalled. `kill -HUP` makes `rsyslog` close and reopen the files it writes to and does not re-read the configuration, so a rule added since the daemon started has no effect until the daemon itself is restarted. That distinction matters again in Part 3, for the opposite reason.

```
# 113_L7_LAB_db: restart, then send three messages that differ in one field each
killall -x rsyslogd; sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
logger -p auth.err      "A"
logger -p auth.info     "B"
logger -p daemon.warning "C"
```

Question 2. Before you look: say which of the three messages A, B and C you expect /var/log/auth-warn.log to hold, and give the reason for each one you exclude. Then read the file and say whether you were right. Say also which file, or files, message A ended up in.

Question 3. /var/log/messages on this machine already held sshd lines before you ran logger at all, and you did not configure sshd to write anything. Name the interface sshd uses to hand its messages to rsyslog, and say what would become of those messages if rsyslogd were not running when sshd produced them.

Everything so far is on db's own disk. That is the arrangement the rest of the lab replaces.

Part 2: sending a copy somewhere else

Forwarding has two halves and neither works alone. The collector has to bind a port and accept connections on it, and each source machine has to be told an address to send to. This part builds both, watches the result on the wire, and changes the format the messages are sent in, which turns out to decide something the last part of the lab depends on.

Start with the collector, because a source machine configured first would spend the interval sending to a port nothing is listening on.

```
# 113_L7_LAB_collector: the log server
docker exec -it 113_L7_LAB_collector bash
```

```
# 113_L7_LAB_collector: load the TCP input module and bind it
cat > /etc/rsyslog.d/10-receive.conf <<'EOF'
module(load="imtcp")
input(type="imtcp" port="514")
EOF
chmod 644 /etc/rsyslog.d/10-receive.conf
killall -x rsyslogd; sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
netstat -tln | grep 514
```

Now each of the three source machines. The rule is a selector and a destination, exactly like the one you wrote in Part 1, except that the destination is a host rather than a file. One @ means send over UDP and two mean send over TCP:

```
# 113_L7_LAB_db: and repeat on 113_L7_LAB_admin and 113_L7_LAB_web
cat > /etc/rsyslog.d/50-forward.conf <<'EOF'
*.* @113.0.0.10:514
EOF
chmod 644 /etc/rsyslog.d/50-forward.conf
killall -x rsyslogd; sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
```

Do that on all three, then send a message from db and read the collector's own file:

```
# 113_L7_LAB_db
logger -p local5.notice "hello from db"
```

```
# 113_L7_LAB_collector
tail -5 /var/log/messages
```

The message is there, in the collector's own file, mixed in with the collector's own messages. Part 3 is about giving it a file of its own.

Definition: PRI

The first thing in a syslog message on the wire: the facility and the severity packed into one number and wrapped in angle brackets. RFC 5424 section 6.2.1 gives the arithmetic as $facility \times 8 + severity$. So a `daemon` (3) message at `err` (3) is $3 \times 8 + 3 = 27$ and appears as `<27>`.

Read one off the wire. Capture on the collector, and send from `db` while the capture runs:

```
# 113_L7_LAB_collector: leave this running, it stops on its own after 15 seconds
timeout 15 tcpdump -n -i 113-S1 -A tcp port 514
```

```
# 113_L7_LAB_db: run this from a second terminal while the capture is going
logger -p local5.notice "read my PRI"
```

Question 4. Quote the **PRI** from your capture. Give the facility number and the severity number it encodes, show the arithmetic that recovers each from the value, and give the name of each. Then say what PRI a message at `auth.crit` would carry.

The rest of that line is the format `rsyslog` forwards in by default: the older BSD layout described in RFC 3164, which is a month, a day, and a wall-clock time to the second, then the hostname, then the program and the message. It carries no year, no timezone and no fraction of a second. `rsyslog` can send the newer format from RFC 5424 instead, and the built-in template for it is called `RSYSLOG_SyslogProtocol23Format`. Change one of the machines over:

```
# 113_L7_LAB_db: the same forwarding, in the other format
cat > /etc/rsyslog.d/50-forward.conf <<'EOF'
**  action(type="omfwd"
        target="113.0.0.10" port="514" protocol="tcp"
        template="RSYSLOG_SyslogProtocol23Format")
EOF
chmod 644 /etc/rsyslog.d/50-forward.conf
kill -x rsyslogd; sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
```

Capture again the same way and send another message.

Question 5. Put the two captured lines side by side. Name three things the RFC 5424 line carries that the RFC 3164 line does not, quoting the value of each from your own capture.

Question 6. A program can produce two messages inside the same second, and `sshd` does: it records one line for a rejected password and another for the connection closing straight

afterwards. Say whether the RFC 3164 line lets a reader at the collector put two such messages in order, whether the RFC 5424 line does, and give the reason in terms of the field that differs between your two captures. Then name what the collector could still use to order them if every sender were sending RFC 3164.

Now make the same change on `admin` and `web`, so all three send RFC 5424.

Two experiments before leaving this part. First, the transport. Change `db` back to the selector form with a single `@`, restart, capture `udp port 514` instead of `tcp`, and send a message.

Question 7. Say whether the datagram left `db`, quoting the line from your capture, and whether the message reached the collector's file. Explain the combination in terms of what the collector is configured to accept, and say what `db` was told about the outcome.

Put `db` back to the TCP RFC 5424 rule before going on.

Second, an outage. Stop the collector's daemon, produce three messages on `db`, then start the collector again and wait half a minute:

```
# 113_L7_LAB_collector
pkill -x rsyslogd
```

```
# 113_L7_LAB_db
logger -p local5.notice "outage 1"
logger -p local5.notice "outage 2"
logger -p local5.notice "outage 3"
```

```
# 113_L7_LAB_collector: bring it back, wait, then count what arrived
sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
sleep 45
grep -c outage /var/log/messages
```

```
# 113_L7_LAB_db: and count what db still has
grep -c outage /var/log/messages
```

The wait is not padding. A forwarding rule that cannot reach the collector is suspended, and it is retried only after `action.resumeInterval`, which is 30 seconds by default and grows as retries keep failing. Anything you send in that interval is handled the same way as the three above.

Question 8. Give the number of the three that reached the collector, and the number still in `db`'s own `/var/log/messages`. TCP acknowledges every byte it delivers on an open connection; say what that does and does not guarantee about a message produced while the collector was unreachable, and name where the copy of those messages does still exist.

Part 3: a file per machine, and keeping it from growing without limit

The collector is now writing everything it receives into its own `/var/log/messages`, next to what the collector itself produces. Three machines' messages in one file, interleaved with a

fourth machine's, is a file you can only search; you cannot look at one machine's day. This part gives each source machine a file of its own, records the fields that the default layout throws away, and then bounds how large those files get.

Definition: Dynamic file name

A destination whose path is computed from each message rather than fixed in the configuration. In rsyslog it is a `template()` used as an action's `dynaFile`, and the template can contain the message's own fields: `%HOSTNAME%` in the path means each sending machine gets a file named after itself, and a machine the collector has never heard from gets one the first time it sends something.

A **dynamic file name** gives each machine its own file. Two more things need to be true at once. The messages that arrive over TCP should go to those files, and the collector's own messages should not, because a collector that files itself under its own hostname alongside the machines it collects has quietly become one of the machines it collects. A `ruleset()` bound to the input is what separates them: everything arriving on that input is evaluated against that ruleset and nothing else is.

```
# 113_L7_LAB_collector: replace the file you wrote in Part 2
cat > /etc/rsyslog.d/10-receive.conf <<'EOF'
module(load="imtcp")
input(type="imtcp" port="514" ruleset="fromhosts")

template(name="perhost" type="string"
         string="/var/log/remote/%HOSTNAME:::secpath-replace%.log")

template(name="detail" type="string"
         string="%timegenerated:::date-rfc3339% %timereported:::date-rfc3339% \
         %HOSTNAME% %syslogfacility-text%.%syslogseverity-text% %syslogtag%msg%\n")

ruleset(name="fromhosts") {
    action(type="omfile" dynaFile="perhost" template="detail")
}
EOF
chmod 644 /etc/rsyslog.d/10-receive.conf
kill -x rsyslogd; sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
```

Send a message from each of the three machines and look at what appeared:

```
# 113_L7_LAB_collector
ls -la /var/log/remote/
tail -2 /var/log/remote/db.log
```

The `detail` template records two timestamps and not one. `%timereported%` is the time the sending machine put in the message; `%timegenerated%` is the time this collector read it off the socket. They are close together and they are not the same value, and which one answers a question depends on the question.

Question 9. No file named after the collector itself appeared under `/var/log/remote/`, although the collector produces messages of its own the whole time. Name the single element of the configuration above that keeps them out, and say what the directory would hold instead if the `action()` had been written outside any `ruleset()`.

The `secpath-replace` on `%HOSTNAME%` is not decoration. `HOSTNAME` is a field the sending machine fills in and this template pastes it into a filesystem path, so without the property option the sender chooses where the collector writes; `secpath-replace` substitutes every path separator in the value with an underscore.

Question 10. A machine on this segment sets its hostname to `../../../../etc/periodic/15min/x` and starts forwarding. Give the exact path the `perhost` template would produce without `secpath-replace`, and the path it produces with it. Then say what the collector's daemon would be doing on that machine's behalf in the first case.

Definition: Log rotation

Renaming or emptying a log file on a schedule so it does not grow without limit, keeping a fixed number of older copies and deleting the oldest. `logrotate` is the tool that does it; its policy files live in `/etc/logrotate.d/` and `rotate 3` means keep three older copies. It runs from a scheduler in normal use, and `logrotate -f` forces a run now, which is how this lab exercises it.

Definition: Inode

The kernel's actual handle on a file's contents, separate from any name pointing at it. Opening a file gives a program a reference to the inode, not to the name, so renaming or deleting the name afterwards does not disturb the program: it goes on reading and writing the same contents, now reachable under whatever name the file has, or under none.

Write a rotation policy for the per-machine files. Start with the plainest form there is:

```
# 113_L7_LAB_collector
cat > /etc/logrotate.d/remote <<'EOF'
/var/log/remote/*.log {
    rotate 3
    size 4k
    missingok
    notifempty
    compress
    delaycompress
}
EOF
chmod 644 /etc/logrotate.d/remote
```

The `chmod` is not optional. `logrotate` refuses to read a policy file that is group- or world-writable and skips it with a message about a bad file mode, and the mode a freshly created file gets here depends on the Docker daemon this lab is running on. `delaycompress` leaves the first rotated copy uncompressed until the next rotation; keep it, for a reason the next step makes visible.

Force a rotation, then send a message and find out where it went:

```
# 113_L7_LAB_collector
logrotate -f /etc/logrotate.d/remote
ls -la /var/log/remote/
```

```
# 113_L7_LAB_db
logger -p local5.notice "after the first rotation"
```

```
# 113_L7_LAB_collector
ls -la /var/log/remote/
grep -l "after the first rotation" /var/log/remote/db.log* 2>/dev/null
```

Question 11. Name the file your message landed in, and say whether `/var/log/remote/db.log` exists at this point.

That is the **log rotation** working exactly as written and the result being wrong. The collector had the file open when `logrotate` renamed it; an open file refers to an **inode** and not to a name, so nothing about the rename reached the collector and it went on appending to the same contents under their new name. Nothing recreates the old name either, because the policy has no `create` directive and `logrotate` does not add one.

You can watch the collector holding it. The kernel lists every file a process has open under `/proc`, and shows each one under the name it currently has:

```
# 113_L7_LAB_collector
ls -l /proc/$(cat /var/run/rsyslogd.pid)/fd/ | grep remote
```

The fix is to tell the collector to let go and reopen by name, which is what `SIGHUP` means to `rsyslog`. Put it in a `postrotate` block so it happens on every rotation:

```
# 113_L7_LAB_collector
cat > /etc/logrotate.d/remote <<'EOF'
/var/log/remote/*.log {
    rotate 3
    size 4k
    missingok
    notifempty
    compress
    delaycompress
    sharedscripts
    postrotate
        kill -HUP $(cat /var/run/rsyslogd.pid) 2>/dev/null || true
    endscript
}
EOF
chmod 644 /etc/logrotate.d/remote
```

`sharedscripts` runs the block once for the whole set rather than once per file, which is what you want for a signal: three files each sending their own HUP is three signals for one piece of work.

Start from a clean collector, because the current one is writing into a file with no live name, then rotate again:

```
# 113_L7_LAB_collector
rm -f /var/log/remote/* /var/lib/logrotate.status
pkill -x rsyslogd; sleep 1; rm -f /var/run/rsyslogd.pid; rsyslogd
```

```
# 113_L7_LAB_db
logger -p local5.notice "before the second rotation"
```

```
# 113_L7_LAB_collector
logrotate -f /etc/logrotate.d/remote
ls -l /proc/$(cat /var/run/rsyslogd.pid)/fd/ | grep remote
```

The descriptor is still open on the rotated file. Wait about ten seconds and run that `ls -l` again, and it is gone: `rsyslog` acts on a HUP from its main loop rather than inside the signal handler, so the signal and the effect are a few seconds apart. Then send another message:

```
# 113_L7_LAB_db
logger -p local5.notice "after the second rotation"
```

```
# 113_L7_LAB_collector
ls -la /var/log/remote/
grep -l "after the second rotation" /var/log/remote/db.log* 2>/dev/null
grep -l "before the second rotation" /var/log/remote/db.log* 2>/dev/null
```

Question 12. Name the file each of the two messages landed in. Then explain what the HUP changed, in terms of what the collector was holding and what it does with the file name afterwards, and say what happens to a message that arrives in the few seconds between the rotation and the collector acting on the signal.

There is a second approach that needs no signal at all. `copytruncate` makes `logrotate` copy the contents aside and then empty the original in place, so the inode the collector is holding is the inode that stays live and the name never moves. Swap the `postrotate` block for it and rotate once more:

```
# 113_L7_LAB_collector
cat > /etc/logrotate.d/remote <<'EOF'
/var/log/remote/*.log {
    rotate 3
    size 4k
    missingok
    notifempty
    compress
    delaycompress
    copytruncate
}
EOF
chmod 644 /etc/logrotate.d/remote
logrotate -f /etc/logrotate.d/remote
```

```
# 113_L7_LAB_db
logger -p local5.notice "after the third rotation"
```

```
# 113_L7_LAB_collector
grep -l "after the third rotation" /var/log/remote/db.log* 2>/dev/null
```

Question 13. Both approaches now leave the next message in the live file, and each opens an interval in which it is not doing what it looks like it is doing. Name the two operations that bound the interval for each approach, say what becomes of a message written inside each one, and say which of the two you would choose for a file whose completeness is the point.

Put the `postrotate` form back before going on. It is the one this lab treats as finished, for the reason your last answer gives.

```
# 113_L7_LAB_collector
cat > /etc/logrotate.d/remote <<'EOF'
/var/log/remote/*.log {
    rotate 3
    size 4k
    missingok
    notifempty
    compress
    delaycompress
    sharedscripts
    postrotate
        kill -HUP $(cat /var/run/rsyslogd.pid) 2>/dev/null || true
    endscript
}
EOF
chmod 644 /etc/logrotate.d/remote
```

Run Status now. Every stage but the incident should be passing before you go on, and the incident action will refuse to run until they are.

Part 4: the incident

You have built the arrangement. This part finds out what it is worth.

Choose **Run the incident** in the TUI, or run `scripts/incident.sh`. It opens real SSH sessions between the machines and makes real HTTP requests, and it finishes by deleting `/var/log/messages` on one of them. It checks first that the collector is recording that machine, and stops without deleting anything if it is not.

When it finishes, look at the machine it happened to:

```
# 113_L7_LAB_db
cat /var/log/messages
```

Every answer below comes from the collector. Nothing on `db` will help you.

```
# 113_L7_LAB_collector: everything the three machines sent, live files and rotated \
copies
ls -la /var/log/remote/
grep sshd /var/log/remote/db.log
```

If a file has been rotated since the incident, the lines you want may be in `db.log.1` rather than `db.log`; read both.

Question 14. Give the number of failed password attempts recorded against a single account on `db`, the name of that account, and the address every one of them came from. Then give the line that ends the sequence, and say what it establishes that the failures alone do not.

Question 15. `db`'s file records two accepted passwords for the same account, from two different addresses. Name which machine each address belongs to. One of the two is ordinary administrative work and the other is not; say which, and name the evidence in the file that distinguishes them, without relying on the address by itself.

Question 16. `admin`'s file records one session opened to it, from an address that opens none in the ordinary course of this network. Give that address and the account, then give the two lines, one from each of two different files, whose timestamps establish which of the two events happened first. Quote the timestamp you are using from each line and say which of the two timestamp fields in the template you took it from.

Question 17. `web`'s file holds lines from `lighttpd` and `db`'s holds lines from `sshd`, and the two carry different facilities. Give both facilities as your files record them. Then say what an investigator can do with a file that records the facility beside every line that they cannot do with the layout `db` used for its own file in Part 1.

Question 18. `db`'s own `/var/log/messages` is gone, and you can still answer the four questions above. Name the point in time at which the copy you are reading was made, relative to the deletion. Then name what somebody who wanted those four answers destroyed would have had to reach in addition to `db`.

Question 19. Compare what root on `db` can destroy with what root on the collector can. Name one property of this evidence that survives the first and does not survive the second, and name one change to this design, not built in this lab, that would narrow that gap.

Checks: what "done" means

Run Status. It sends a message from each of the three source machines and reports, for each one, whether it reached the collector, whether it landed in a file named after that machine, and whether the hostname, facility and severity were recorded beside it. It then reports whether the collector still holds the incident's authentication lines and whether `db`'s own log file is gone.

The lab is finished when every check passes: three machines forwarding, three files under `/var/log/remote/` and none for the collector itself, a rotation policy `logrotate` will read, and the incident's evidence readable on the collector after the machine it happened to lost its copy.

Safety note

This lab runs on one Open vSwitch bridge with no route off the host. The four containers can reach each other and nothing else, and nothing they run is aimed anywhere but at each other. The SSH sessions the incident opens are between two containers this lab created, using accounts this lab created, on a segment that exists only while the lab is spawned.

Teardown

Remove the lab: choose **Teardown** in the TUI, or run `scripts/teardown.sh`. It deletes this lab's containers and nothing else, and the log files go with them.